

Lazarus

An AI Pipeline for Mining, Matching, and Verifying Science's Failures and Orphaned Methods

Accelerating Scientific Discovery with Intelligence Research Technology

Proposal and Technical Design Document

2026

Lazarus treats science's unpublished failures and orphaned cross-field methods as an underused resource rather than a footnote. A mining layer extracts explicit negative-result and retraction signals from the literature and tags why each attempt failed; a matching layer embeds papers by underlying methodological structure rather than topic, surfacing techniques from unrelated fields that share a hidden shape with a stuck problem; and a verification layer scores the confidence of both kinds of output, routing anything uncertain to lightweight human review before it reaches a researcher. The result is a system that resurrects what the literature has already tried and quietly abandoned, and tells a researcher, with calibrated confidence, which parts are actually worth reviving.

Introduction

This document proposes Lazarus, developed in response to the theme “Accelerating Scientific Discovery with Intelligence Research Technology.” The theme was issued as a title only, with no dataset, problem statement, or judging rubric attached, which means the first task was not building a system against a specified scoring function but deciding, deliberately, what the theme is actually asking for.

The default reading of “accelerating scientific discovery” with AI is to make researchers faster at tasks they already do — searching literature, summarising papers, drafting reviews — and a crowded field of existing tools, including Elicit, Consensus, Scite, and SciSpace, already serve exactly this function well. Lazarus takes a different reading: acceleration comes not from doing existing research tasks faster, but from not re-doing work that has already been attempted and quietly abandoned, and from noticing when two fields, unaware of each other, have already solved each other's open problems.

The system that follows from this reading has three stages. A mining stage treats the literature's negative results — retracted papers, withdrawn preprints, abandoned public repositories — as a resource in their own right, extracting not just the fact of failure but its cause. A matching stage embeds papers by the shape of their underlying method rather than by topic, so a stalled problem in one field can be matched against a technique from an entirely unrelated field that shares its structure. A verification stage scores the confidence of both mining and matching output and routes anything below threshold to lightweight human review, so the system's claims are calibrated rather than merely plausible-sounding. Each stage is a real, separately defensible piece of engineering; together they describe a system whose value compounds with the size and diversity of the literature it draws on.

The remainder of this document sets out the alignment with the assigned Frontier Technologies and Emerging Innovation category, the vision and scope chosen for the open theme, the problem the team has defined for itself in the absence of a stated one, the objectives the project pursues, the architecture and detailed functional design of each stage, the technology stack and data strategy behind them, the target audience, a budget proposal and financial outlook within the RM 100,000 ceiling available, the development challenges and risks expected honestly, and a phased roadmap for delivery.

Motivation

The first motivation is structural, not incidental: publication bias means the scientific record is overwhelmingly a record of what worked, while what failed — and specifically why — is almost never written down in a form future researchers can search. Expensive, well-intentioned effort is spent re-discovering dead ends that another lab, another country, or another decade already walked into and quietly abandoned. This is not hypothetical: retraction databases, withdrawn preprints, and abandoned public repositories already contain this signal, unindexed and unsearched, at a scale a mining pipeline can meaningfully draw on.

The second motivation is that genuine breakthroughs disproportionately come from analogy across fields that do not read each other's literature, and this is presently left to chance and to the rare researcher who happens to have read both. A matching system that deliberately searches for structural rather than topical similarity manufactures the kind of cross-pollination that currently depends on personal reading habits, at a scale no individual researcher could replicate alone.

The third motivation is a trust problem specific to this category of tool: an AI system claiming “this failed” or “these are analogous” is a claim a researcher will act on, and a wrong claim wastes exactly the time the system exists to save. This is why verification is a first-class stage of the architecture rather than an afterthought — the system's claims are only as valuable as a researcher's confidence that they are calibrated.

Alignment with Frontier Technologies and Emerging Innovation

Lazarus is submitted under the Frontier Technologies and Emerging Innovation category, and the fit is substantive rather than nominal. The frontier-technology component is the contrastive structural-embedding model described in the Lazarus Concept section: training embeddings to capture methodological rather than topical similarity is not an application of an existing off-the-shelf technique, but a genuinely unsolved machine-learning problem this project takes on directly, rather than a thin AI wrapper over an existing service. The emerging-innovation component is the reframing itself: treating a scientific literature's negative results as a first-class, minable asset, and treating cross-field analogy as something to manufacture deliberately rather than leave to chance, is a discovery-acceleration mechanism that existing research-assistant tools do not attempt, precisely because most of them are built to summarise and retrieve confirmed, published work rather than question what the published record leaves out.

Community Impact and Contribution

Lazarus is designed to create impact beyond the team that builds it, at the level of the individual researcher, the institution, and the broader research ecosystem.

At the individual level, the platform gives any researcher, regardless of how well-resourced their lab or how wide their personal reading habits, access to a check against redundant failure and a systematic search for cross-field analogy that would otherwise depend on luck, seniority, or an unusually well-connected supervisor. This is particularly meaningful for researchers at less well-funded institutions, where the cost of repeating an already-failed experiment is proportionally far more damaging than it is for a well-funded lab that can absorb the loss.

At the institutional level, universities and research offices that adopt Lazarus gain a searchable institutional memory of failure that would otherwise be lost every time a graduate student graduates or a postdoc moves on, taking their private knowledge of what didn't work with them. This addresses a chronic knowledge-loss problem in academic research that has nothing to do with any individual's competence and everything to do with the absence of a system built to retain it.

At the level of the research ecosystem, a platform that makes negative results and cross-field analogy systematically discoverable contributes, in a small but real way, to correcting the publication-bias distortion described in the Problem Statement: work that would otherwise vanish without a trace becomes a permanent, searchable part of the record, and connections between fields that would otherwise never be made become findable by any team using the platform, not just the one lucky enough to notice.

Novelty of the Project

The novelty of Lazarus rests on two claims that are each independently verifiable against the existing landscape of research-AI tools, rather than a single, harder-to-defend claim of overall originality.

The first is architectural: no existing literature-assistant tool treats negative results, retractions, withdrawn preprints, and abandoned repositories, as a first-class object of retrieval in their own right. Existing tools retrieve, summarise, and rank published, confirmed work; Lazarus is built specifically to retrieve the record's failures, which requires a different extraction and classification pipeline entirely, not a configuration change to an existing paper-search tool.

The second is technical: the structural embedding model described in the Lazarus Concept section is fine-tuned via contrastive learning specifically to separate methodological similarity from topical similarity, an axis that off-

the-shelf semantic embeddings, including those behind every major literature-search tool named in the Introduction, are not built to capture. This is a genuine machine-learning contribution rather than an application of an existing embedding API, and it is the single hardest, most defensible technical claim in this proposal.

Combined, these two claims describe a system whose novelty is not a matter of framing or presentation, but of doing something existing tools are not built to do: retrieving what the literature has forgotten, and connecting what the literature has never linked, rather than doing something existing tools already do somewhat better or worse.

Vision

The dream behind Lazarus is a scientific literature that never actually loses anything it learns, including its failures. In this vision, no research group anywhere spends six months rediscovering that a method fails under conditions someone else already documented and abandoned three years ago in a different journal, a different country, or a different field entirely. Every retraction, every withdrawn preprint, every quietly shelved repository becomes a permanent, searchable contribution to the collective effort rather than a private, forgotten cost absorbed by one lab. At the same time, no promising cross-field connection goes unmade merely because the two researchers who could have seen it never read each other's journals. Lazarus's long-run ambition is to become the layer of scientific infrastructure that makes the literature's failures as valuable as its successes, and stops the accident of departmental boundaries from deciding which ideas ever meet.

At scale, this means a researcher beginning a new line of work checks Lazarus the way a lawyer checks case law: not as a novelty, but as due diligence that would be professionally embarrassing to skip. It means research funders and journals can eventually treat a null result logged through the system as a citable contribution in its own right, closing part of the publication-bias gap that currently discards most experimental effort as if it never happened.

Scope

Given the absence of a specified problem statement, this section states explicitly what the Lazarus prototype does and does not attempt, so the concept is judged against a scope the team has deliberately chosen rather than an unbounded promise.

In Scope

- A mining pipeline that classifies and tags failure records from a bounded, openly licensed set of sources: a defined retraction database, a snapshot of preprint withdrawal notices, and abandoned public repositories linked to a paper
- A structural embedding model, fine-tuned via contrastive learning, demonstrated on a deliberately bounded set of fields chosen for open metadata availability
- A verification layer combining automated confidence scoring with a lightweight human review queue staffed by a small recruited reviewer pool
- A working demonstration interface allowing a researcher to query a stated problem and receive ranked failure records and cross-domain matches with visible confidence indicators
- A costed budget proposal and a conservatively modelled financial outlook within the RM 100,000 ceiling

Out of Scope for This Phase

- Comprehensive, real-time coverage of the entire scientific literature; the prototype works from bounded, periodically refreshed snapshots, not a live crawler

- Automating the actual adaptation of a cross-domain method into a researcher's own field; the system proposes the analogy, the researcher performs the transplant
- A general-purpose task marketplace for scientific labour beyond the Verification Exchange's narrow reviewing role
- Wet-lab or physical-science experimental validation of any kind; the system operates entirely on literature and metadata
- Formal partnerships or paid licensing arrangements with academic publishers for paywalled full-text access

Good to Add, Not Core

- A browser extension that checks a researcher's in-progress draft against the Graveyard automatically
- A public API tier allowing institutional customers to integrate results ahead of the enterprise tier's formal launch
- A researcher feedback mechanism that improves match ranking over time based on which cross-domain suggestions researchers report as useful

Problem Statement

Read literally, “Accelerating Scientific Discovery with Intelligence Research Technology” does not specify a deliverable, an audience, or a dataset. It specifies a direction. This proposal treats the theme as requiring the team to identify, argue for, and scope a real problem before proposing a system to solve it. Four interconnected problems, each already well documented in the research-methods and science-of-science literature, define the gap Lazarus is designed to address.

First, the scientific record has a severe and well-known negative-results gap. Journals overwhelmingly publish confirmatory, positive findings, while null results, failed replications, and abandoned approaches are rarely written up in any citable, searchable form, even though the effort spent producing them was real. A retraction or a withdrawn preprint occasionally makes this failure visible, but even then it is indexed as bibliographic metadata, not as a substantive lesson a future researcher could search against before repeating the same mistake.

Second, the scientific literature is organised by journal and by field in a way that actively suppresses cross-domain analogy. A technique that solves a stuck problem in one field may already exist, solved, in an unrelated field, described in language and venues that a researcher elsewhere has no professional reason to ever encounter. Existing search and recommendation tools cluster papers by topical similarity, which reinforces field boundaries rather than crossing them.

Third, the AI tools that have entered scientific research assistance in recent years are overwhelmingly built around summarising and retrieving what has already been published and verified, which means they inherit and often amplify the positive-results bias of the literature itself rather than correcting for it. A researcher using a state-of-the-art literature assistant today still has no systematic way to ask whether an approach has already failed for someone else.

Fourth, where AI systems do venture into flagging failures or proposing analogies, the central risk is confident-sounding but ungrounded output. An AI claim that an approach “has already failed” or that two methods “are structurally analogous” is the kind of claim a researcher will act on, and an ungrounded version of either claim is actively worse than no claim at all. Any system operating in this space has an obligation to be honest about its own confidence that goes beyond what most current research-assistant tools attempt.

Summary of the Problem Statements

Taken together, these four problems describe a scientific ecosystem that systematically discards its own failures, rarely connects methods across field boundaries, and is now acquiring AI tools that reinforce rather than correct these two blind spots, without the confidence-calibration discipline that operating safely in this space requires. Lazarus is proposed as a direct, three-stage response to this specific gap, rather than as a further entrant into the already well-served category of literature summarisation and retrieval.

Project Objectives

In response to the problems identified above, the project pursues five interconnected objectives.

To Make Science's Negative Results Searchable and Actionable

The first objective is to convert the literature's scattered, unindexed failure signal into a structured, searchable resource that tags not only that an approach failed but why, so a researcher beginning new work can check whether it has already been tried and abandoned.

To Systematically Surface Cross-Domain Methodological Analogies

The second objective is to replace chance-based cross-field discovery with a deliberate matching process, embedding research by the structure of its underlying method rather than by topic.

To Keep Every AI Claim Calibrated and Verifiable

The third objective is to ensure that every failure tag and every cross-domain match carries an honest, calibrated confidence score, with anything below a defensible threshold routed to lightweight human review.

To Demonstrate a Financially Sustainable Path Beyond the Prototype

The fourth objective is to show that Lazarus has a credible path to standalone financial sustainability through a subscription model capable of supporting continued development after the initial funding period.

To Prove the Concept on a Bounded, Defensible Corpus

The fifth objective is to demonstrate the full three-stage pipeline convincingly on one deliberately bounded corpus, rather than to claim, without evidence, that the system already works across the entirety of scientific literature.

Summary of the Project Objectives

Collectively, these objectives describe a system that treats scientific failure as an asset, treats field boundaries as an engineering problem rather than a fact of nature, holds itself to a higher confidence-calibration standard than most current research-AI tools, and is scoped and costed as a real, financially grounded proposal rather than a hackathon novelty.

The Lazarus Concept

The Core Pipeline — Mine, Match, Verify

Lazarus is built around three sequential stages that share one underlying corpus and one verification discipline. The Mining stage ingests retraction databases, preprint withdrawal notices, and abandoned public repositories, classifies each as a genuine negative result, and extracts a structured failure record describing the method, the

dataset, and the likely cause of failure. The Matching stage embeds every record, both the working literature and the mined failures, in a representation trained to capture methodological structure rather than topic, and retrieves cross-domain candidates whose underlying approach resembles a researcher's stated problem. The Verification stage scores the confidence of every mining tag and every matching candidate, surfacing high-confidence output directly and routing lower-confidence output to a lightweight review queue before it reaches a researcher.

The Differentiator — Structural Rather Than Topical Embedding

The component that distinguishes Lazarus from existing literature-assistant tools is the embedding model behind the Matching stage. Off-the-shelf semantic embeddings cluster papers by subject matter, which is precisely the axis along which cross-domain analogies are invisible. Lazarus instead fine-tunes embeddings using contrastive pairs constructed so that papers sharing an underlying method, regardless of field, are pulled together in vector space, while papers sharing only a topic are not artificially pulled together on that basis alone. This is the one genuinely hard machine-learning claim in the proposal, and it is treated as the centrepiece of the technical build rather than a secondary feature.

The Module Map

The pipeline is exposed to a researcher through three corresponding modules. The Graveyard is the search interface over mined failures, letting a researcher check a proposed approach against everything the system already knows has failed. The Citation Necromancer is the cross-domain analogy interface, taking a stated problem and returning ranked, structurally-matched candidates from unrelated fields. The Verification Exchange is the confidence layer running beneath both, visible to the researcher only as a confidence indicator and, where relevant, a note that a given result has been human-confirmed.

Detailed Functional Description of Application Modules

The Graveyard

The Graveyard ingests structured and semi-structured failure signal from three source types: formal retraction notices, preprint withdrawal records, and abandoned public code repositories associated with a research project. Each source type carries a different signal quality and requires a different extraction approach. Retraction notices are the most structured of the three, since journals and indexing services typically require a stated reason at the time of retraction, though that reason is often written in defensive, legalistic language that must itself be normalised before it is useful, for example distinguishing an honest methodological error from a formatting or authorship dispute that has nothing to do with the underlying science. Withdrawal notices from preprint servers are sparser and far less standardised: a withdrawal note might say nothing more than “withdrawn at author's request,” which requires the classifier to fall back on secondary signals, such as whether a corrected or superseding version was later posted, whether the withdrawal followed public comment, and how long the preprint remained live before withdrawal. Abandoned repositories are the noisiest of the three signals by a wide margin, since a repository can go quiet for many reasons that have nothing to do with the underlying approach failing, a student graduating, a grant ending, a researcher changing institutions, and the abandonment classifier is trained specifically to separate these benign explanations from genuine technical dead ends, using signals such as the shape of the commit history immediately before activity stopped, whether the associated paper, if any, was itself withdrawn or never published, and explicit abandonment language surfaced in README files, issue threads, or commit messages themselves.

Once classified, each failure record is tagged along a small, fixed taxonomy of failure causes, for example methodological flaw, insufficient effect size, data availability collapse, or superseded by a better approach, so that

records are comparable and filterable rather than free text a researcher must read in full. Where a source record plausibly fits more than one cause, for instance a result that failed to replicate specifically because the original dataset became unavailable, the record is tagged with its primary cause and a secondary cause where the source material supports it, rather than being forced into a single artificial category. A researcher interacts with the Graveyard by describing a planned approach in natural language, for example “using a linear probing classifier to detect X from Y sensor data,” and receiving a ranked list of matching failure records, each showing the failure cause, the classification confidence, a short paraphrased excerpt of the original stated reason where one exists, and a link to the original source so the researcher can verify the context themselves rather than relying solely on the system's summary.

The prototype scopes the Graveyard to a single bounded corpus, drawn from an openly licensed retraction database and a defined snapshot of preprint withdrawal records, rather than attempting comprehensive real-time coverage of the scientific record, a boundary the Scope section states explicitly and the Data Strategy section explains further. As the Verification Exchange confirms or rejects borderline classifications over time, those confirmed decisions are folded back into the training signal for the classifier itself, so the Graveyard's precision is expected to improve gradually with usage rather than being fixed at whatever the initial classifier achieves at launch.

Core Functions

- Ingest and classify negative-result signals from retraction databases, preprint withdrawals, and abandoned repositories
- Tag each classified failure record against the fixed failure-cause taxonomy
- Serve natural-language queries against the classified failure corpus
- Surface confidence status and source provenance for every returned record

Internal Flow

1. Scheduled Ingestion: Vercel Cron Jobs trigger the ingestion pipeline against the retraction database, preprint withdrawal snapshot, and repository-abandonment sources on a recurring schedule.

2. Classification: Each newly ingested record is classified by the Gemini API against the fixed failure-cause taxonomy, together with a confidence score from the Verification Exchange.

3. Confidence Routing: Records above the confidence threshold are marked system-verified and added to the searchable corpus; records below threshold are placed in the reviewer queue.

4. Query Resolution: When a researcher submits a proposed approach, the query is matched against the classified corpus, and matching records are ranked and returned with their failure cause, confidence status, and source link.

How It Runs in the Application

A researcher opens the Graveyard tab, describes a planned approach in the query field, and receives a ranked list of matching failure records within seconds, each showing the failure cause, a confidence badge, and a link to the original retraction or withdrawal notice.

Module Interactions

The Graveyard shares its classification pipeline and confidence scoring with the Verification Exchange, and shares its underlying corpus storage in Postgres with the Citation Necromancer, though the two modules query that storage differently: the Graveyard by classified failure cause, the Citation Necromancer by structural embedding distance.

The Citation Necromancer

The Citation Necromancer takes a researcher's stated problem, described in natural language, for example “I need to detect gradual drift in a noisy time series without labelled examples of drift,” and returns candidate papers from

fields structurally unrelated to the one the researcher is working in, ranked by how closely their underlying method matches the structure of the stated problem rather than by topical overlap. In this example, a genuinely useful cross-domain match might come from geology, where researchers detect gradual sensor drift in seismic monitoring equipment using a comparable unsupervised change-detection approach, a field a machine-learning researcher would have no professional reason to read.

Both the query and the candidate corpus are projected into the structural embedding space described in the Lazarus Concept section, and retrieval deliberately excludes a configurable radius of topical closeness, measured using a conventional topic embedding run alongside the structural one, so the system cannot simply return more of the same field dressed up as a cross-domain match. This exclusion radius is itself a tunable parameter rather than a fixed rule: set too narrow, the system risks surfacing matches that are cross-domain in name only, drawn from an adjacent sub-field that shares enough vocabulary to sneak past a loose filter; set too wide, the system risks discarding genuinely useful matches from fields that are conceptually close even though institutionally distant. Calibrating this radius is one of the concrete outputs expected from the pilot phase described in the Roadmap section.

Each returned candidate is accompanied by a short, generated explanation of the structural similarity the embedding detected, written in plain language rather than as a bare similarity score, describing specifically which aspect of the method rhymes, for instance that both approaches reduce the problem to detecting a slow shift in an otherwise stationary distribution without ground-truth labels, rather than a vague assertion that the papers are “similar.” A researcher can use this explanation to quickly judge whether the proposed analogy is worth investigating further. The explanation step is treated throughout this proposal as interpretive scaffolding for a human decision, not a substitute for the researcher's own judgement about whether the transplant is actually valid: the system's job ends at proposing a credible lead, and the researcher's domain expertise is what determines whether the borrowed method genuinely transfers to their data, their assumptions, and their constraints.

Core Functions

- Embed a researcher's stated problem using the fine-tuned structural embedding model
- Retrieve structurally similar candidates from unrelated fields, excluding topically close matches
- Generate a plain-language explanation of why a candidate is structurally relevant
- Surface confidence status for every returned match

Internal Flow

1. Query Embedding: The researcher's stated problem is embedded using the fine-tuned structural embedding model described in the Lazarus Concept section.

2. Restricted Retrieval: The embedding is used to search the pgvector index for nearest neighbours, with a configurable exclusion radius removing candidates that are simply topically close rather than structurally similar.

3. Confidence Scoring: Each candidate match is scored by the Verification Exchange; low-confidence matches are routed to the reviewer queue rather than surfaced directly.

4. Explanation Generation: For matches above the confidence threshold, the Gemini API generates a short, plain-language explanation of the detected structural similarity.

5. Ranked Delivery: Candidates are returned ranked by match confidence, each with its explanation and confidence status.

How It Runs in the Application

A researcher opens the Citation Necromancer tab, describes a stuck problem, and receives a ranked list of candidate papers from unrelated fields, each with a short explanation of the structural similarity the system detected and a confidence indicator showing whether the match is system-verified or pending review.

Module Interactions

The Citation Necromancer depends on the same fine-tuned embedding model and Verification Exchange confidence scoring as the Graveyard, and its confirmed matches feed back into the embedding model's periodic re-training, described in the Development Challenges section.

The Verification Exchange

The Verification Exchange is the confidence layer shared by both the Graveyard and the Citation Necromancer, rather than a separate researcher-facing product in its own right. Every failure classification and every cross-domain match is scored for confidence using a combination of signals: model-internal signals such as the classifier's decision margin for a Graveyard tag or the embedding retrieval distance for a Citation Necromancer match, and, for the mining stage specifically, corroboration across multiple independent sources describing the same failure, for example a retraction notice whose stated reason is independently echoed in a subsequent citing paper's discussion section. A record with a wide classifier margin and independent corroboration is treated very differently from a record with a narrow margin and a single, ambiguous source, even if both nominally receive the same failure-cause tag, because the confidence score, not the tag alone, is what determines whether a result reaches a researcher directly or first passes through review.

Output above a defensible confidence threshold is surfaced directly and tagged as system-verified, visible to the researcher through the Confidence Indicator described in the Components Description section. Output below the threshold is placed in a lightweight review queue rather than withheld outright, so a researcher still sees that a plausible result exists, marked as pending, rather than nothing at all. A small pool of subject-matter reviewers, recruited from the target audience itself and most practically drawn from the pilot institutional partners described in the Roadmap section, works through the queue, confirming or rejecting each flagged item with the original source material visible alongside the system's proposed tag or match. A confirmed item is added to the system-verified pool and becomes a labelled training example that improves the underlying classifier or embedding model's future confidence calibration on similar cases; a rejected item is logged as a known false positive, so the same misclassification pattern does not resurface unchecked the next time a structurally similar record is ingested.

This is a deliberately narrow scope for the verification layer: it exists to protect the credibility of the Graveyard's and the Citation Necromancer's output, not to become a general-purpose task marketplace for scientific labour, a broader idea considered and set aside earlier in this project's development, described in the Novelty of the Project section, in favour of this tighter, load-bearing role. The trade-off this narrow scope accepts is throughput: because every below-threshold item requires a real person's time, the size of the reviewer pool is a genuine constraint on how much of the corpus can be surfaced with confidence at any given moment, which is precisely why the confidence thresholds themselves are treated as a tunable, actively managed parameter in the Cost Optimisation Strategy section rather than a fixed setting decided once at launch.

Core Functions

- Score the confidence of every failure classification and every cross-domain match
- Route below-threshold output to a lightweight human review queue
- Present pending items to recruited subject-matter reviewers
- Feed confirmed and rejected decisions back into future confidence scoring

Internal Flow

1. Signal Collection: For every candidate output from the Graveyard or the Citation Necromancer, the Verification Exchange collects model-internal signals, such as classifier margin and retrieval distance, and, for failure records specifically, cross-source corroboration.

2. Threshold Evaluation: These signals are combined into a single confidence score and compared against a defensible threshold.

3. Routing: Output above threshold is tagged system-verified and returned to the requesting module; output below threshold is placed in the reviewer queue.

4. Reviewer Confirmation: A recruited subject-matter reviewer confirms or rejects each queued item through the Reviewer Queue and Verification Interface.

5. Feedback Loop: Confirmed items are added to the system-verified pool; rejected items are logged and excluded, and recurring rejection patterns inform future threshold calibration.

How It Runs in the Application

The Verification Exchange has no researcher-facing interface of its own; it operates beneath the Graveyard and the Citation Necromancer, visible to a researcher only as the confidence badge on each result, and to a recruited reviewer as the review queue described in the Reviewer Queue and Verification Interface component.

Module Interactions

The Verification Exchange is the shared dependency of both other modules and has no independent function without them; its reviewer pool is drawn from the same institutional partners described in the Target Audience and Roadmap sections.

Overall System Execution Summary

In operation, Lazarus runs as a query-driven, verification-gated pipeline rather than a continuously autonomous agent, so a researcher is always the one initiating a check and always the one making the final judgement call.

Query Submission

A researcher submits a query, either a proposed approach to check against the Graveyard, or a stuck problem to match against the Citation Necromancer, through the TypeScript frontend, which performs lightweight client-side validation before the request reaches the backend.

Backend Processing

The FastAPI backend receives the validated request and routes it to the appropriate module. For a Graveyard query, the backend searches the classified failure records in Postgres for matches against the stated approach. For a Citation Necromancer query, the backend embeds the query using the fine-tuned structural embedding model and retrieves nearest neighbours from the pgvector index, excluding the configurable topical-closeness radius described in the Lazarus Concept section.

Confidence Scoring and Routing

Every candidate result, whether a failure record or a cross-domain match, passes through the Verification Exchange's confidence scoring before it is returned. Results above the defensible threshold are tagged system-verified and returned directly. Results below the threshold are placed in the reviewer queue rather than withheld outright, so a researcher sees a clearly marked pending item rather than nothing at all.

Response Delivery and Explanation

For Citation Necromancer results specifically, a short natural-language explanation of the structural similarity is generated via the Gemini API before the result is returned, so a researcher receives an interpretable rationale alongside the ranked candidate rather than a bare similarity score.

Reviewer Confirmation Loop

Items placed in the reviewer queue are confirmed or rejected by the small pool of subject-matter reviewers described in the Verification Exchange's functional description. Confirmed items are added to the system-verified pool and, where the same pattern recurs, inform future confidence scoring; rejected items are discarded and logged, so the same false positive is not repeatedly surfaced.

Ingestion and Corpus Growth

Independently of any researcher's query, the scheduled ingestion jobs described in the Deployment and Operations section pull new retraction, withdrawal, and repository records on a recurring basis, classify them, and add them to the corpus, so the Graveyard's coverage and the Citation Necromancer's match pool grow over time without requiring a manual rebuild.

Target Audience

Lazarus is built around three audience segments, each with a different relationship to the platform.

Individual researchers, principally graduate students, postdoctoral researchers, and principal investigators in computationally tractable fields, are the primary users of the Graveyard and Citation Necromancer interfaces directly, checking proposed approaches before committing time to them and searching for cross-domain leads when a project is stuck. This segment is expected to adopt the platform through a low-cost or freemium individual tier, since individual researchers rarely control discretionary budgets of any size.

University research offices and library systems are the primary institutional purchaser, licensing access on behalf of their researcher population in the same way they already license reference-management and literature-database subscriptions, and represent the main early revenue segment because the purchasing decision is centralised.

Corporate research and development departments, particularly in fields where a failed experimental direction carries a direct, large cost — pharmaceuticals, materials science, and advanced manufacturing prominent among them — are the highest-value long-term segment, since the economic value of avoiding one redundant failed experiment can exceed a full annual subscription many times over. This segment is expected to adopt later, once the mined-failure corpus and the structural-matching quality have reached a scale that makes the tool's recommendations reliable enough for a commercial R&D budget to depend on.

System Architecture

The system is organised into four layers. An Ingestion Layer pulls from retraction databases, preprint withdrawal feeds, and public code repository metadata on a scheduled basis, normalising each source into a common record format. A Modelling Layer holds the failure classifier, the structural embedding model, and the confidence-scoring model, and is the only layer that changes as the underlying models are retrained or improved. A Retrieval and Verification Layer indexes embeddings in a vector store, serves query-time retrieval for both researcher-facing modules, and routes low-confidence output to the reviewer queue. An Application Layer exposes the two researcher-facing modules through a web interface and a documented API, so institutional customers can integrate results into their own tools.

This is deliberately a thinner architecture than a production-grade research platform would eventually need. The prototype's purpose is to demonstrate that the pipeline works end to end on a bounded corpus and that the structural-embedding claim holds up, not to pre-build infrastructure for a scale of usage the project has not yet earned.

Technology Stack

Overall Stack Choice

The prototype is built on an open, swappable stack rather than committing early to a single proprietary provider, given the project's total budget and the uncertainty about which sponsor or partner infrastructure, if any, may become available as the project progresses. The stack is split cleanly along one principle: Python where the work is modelling or data, TypeScript where the work is interface.

Frontend Layer

The researcher-facing interface is built in TypeScript using React, with Next.js providing routing, server-side rendering for the query and results pages, and a thin API-proxy layer to the Python backend. TypeScript's static typing is used deliberately across the request and response shapes for the Graveyard and Citation Necromancer queries, so a schema mismatch between frontend and backend surfaces at build time rather than as a runtime error in front of a researcher.

Backend and API Layer

The backend is written in Python, using FastAPI to expose the Graveyard, Citation Necromancer, and Verification Exchange functionality as a documented REST API. FastAPI's automatic OpenAPI schema generation keeps the TypeScript frontend's request and response types in sync with the backend contract, and this same documented API is what institutional customers integrate against under the Enterprise tier described in the Budget and Return on Investment section.

Modelling and Data Layer

Python is used throughout the modelling and pipeline layers. Failure classification and match-explanation generation use Google's Gemini models, accessed through the Gemini API, with the calling code abstracted behind a provider interface so the underlying model can be swapped for a locally hosted alternative if cost or availability requires it. This is a separate model from the structural embedding model: the embedding model is a smaller, open-weight sentence-embedding model, fine-tuned in-house using contrastive learning, rather than a hosted embeddings endpoint, since the fine-tuning itself, not the strength of the base model, is the project's central technical claim. Retrieved vectors are indexed in Postgres using the pgvector extension, so the same database holds both the relational failure-record metadata and the embedding index, rather than running a separate dedicated vector database at a scale that does not yet need one.

Deployment and Operations

Given the team's preference for operational simplicity, the entire application is consolidated onto Vercel rather than split across multiple hosting providers. The Next.js frontend and the FastAPI backend, deployed as Python serverless functions, both run on Vercel, and the database uses Vercel Postgres, built on Neon, which supports the pgvector extension the Modelling and Data Layer relies on. This gives the team one platform, one dashboard, and one billing relationship covering the frontend, the API, and the database. Vercel Cron Jobs trigger the scheduled ingestion runs; because the prototype's corpus is deliberately bounded, as described in the Scope section, these

runs stay within Vercel's serverless execution limits at this scale, though that boundary is the first thing to revisit, most likely by moving ingestion to a dedicated worker, if the corpus grows past what the bounded prototype assumes. A GitHub Actions pipeline runs tests before every Vercel deploy. This whole layer is sized to the RM 7,000 hosting and deployment line in the budget.

System Components to Be Developed

A. User-Facing Application Components

- 1. Web Application (Next.js and TypeScript Frontend)
- 2. Query and Results Interface
- 3. Confidence Indicator and Explanation Display

B. Core Functional Modules

- 4. The Graveyard Module (Failure Mining and Classification)
- 5. The Citation Necromancer Module (Cross-Domain Structural Matching)
- 6. The Verification Exchange Module (Confidence Scoring and Review Routing)

C. System Intelligence and Data Components

- 7. Structural Embedding Model (Contrastive Fine-Tuning Pipeline)
- 8. Ingestion and Normalisation Component
- 9. Retrieval Component (Postgres and pgvector)

D. Feedback and Review Components

- 10. Reviewer Queue and Verification Interface

E. Infrastructure and Operational Components

- 11. API Gateway and Backend Service Layer (FastAPI)
- 12. AI Integration Component (Gemini API Interface)
- 13. Deployment and CI/CD Pipeline (Vercel and GitHub Actions)

Components Description in Development Stacks

Each component below is described at the level of proportion this proposal is written at: a working understanding of what it does and why it matters, rather than a full build specification. Depth here is deliberately lighter than a production engineering document, consistent with the Scope section's framing of this as a concept and business case rather than a judged build.

1. Web Application (Next.js and TypeScript Frontend)

Function: Provides the researcher-facing entry point to the platform.

Responsibilities: Render the query and results pages; manage client-side validation and routing; maintain type-safe request and response shapes against the backend contract.

Technology: Next.js, React, TypeScript.

Significance: The only surface most researchers and institutional evaluators ever see, so its clarity carries the platform's credibility.

2. Query and Results Interface

Function: Captures a researcher's stated approach or problem and displays ranked results.

Responsibilities: Present the Graveyard and Citation Necromancer query forms; render ranked results with confidence indicators; support saved searches for paid-tier users.

Technology: React components within the Next.js frontend.

Significance: Where the platform's honesty commitment becomes visible, since every result is shown with its confidence status rather than presented as uniform fact.

3. Confidence Indicator and Explanation Display

Function: Surfaces the Verification Exchange's confidence status and, for matches, the generated explanation.

Responsibilities: Distinguish system-verified from pending-review results visually; display the plain-language structural-similarity explanation for Citation Necromancer matches.

Technology: React, consuming the FastAPI backend's confidence and explanation fields directly.

Significance: The component most directly responsible for the False Confidence Risk mitigation described in the Risks and Caveats section.

4. The Graveyard Module (Failure Mining and Classification)

Function: Classifies and tags mined failure records.

Responsibilities: Ingest retraction, withdrawal, and repository-abandonment signals; classify each against the fixed failure taxonomy; expose the query interface described in its functional description.

Technology: Python, the Gemini API for classification, Postgres for storage.

Significance: One half of the platform's core novelty claim, described in the Novelty of the Project section.

5. The Citation Necromancer Module (Cross-Domain Structural Matching)

Function: Matches a stated problem against structurally similar work from unrelated fields.

Responsibilities: Embed queries and corpus records using the fine-tuned structural embedding model; enforce the topical-exclusion radius; generate match explanations.

Technology: Python, the in-house fine-tuned embedding model, pgvector, the Gemini API for explanation generation.

Significance: The other half of the platform's core novelty claim, and its single hardest engineering problem.

6. The Verification Exchange Module (Confidence Scoring and Review Routing)

Function: Scores confidence and routes low-confidence output to human review.

Responsibilities: Combine model-internal signals and source corroboration into a confidence score; route below-threshold output to the reviewer queue; log reviewer decisions.

Technology: Python, Postgres for the review queue.

Significance: Protects the credibility of the other two modules' output and is the direct answer to the False Confidence Risk.

7. Structural Embedding Model (Contrastive Fine-Tuning Pipeline)

Function: Produces the embeddings the Citation Necromancer depends on.

Responsibilities: Construct the contrastive training set described in the Development Challenges section; fine-tune an open-weight sentence-embedding model; re-train periodically as the corpus and confirmed matches grow.

Technology: Python, an open-weight sentence-embedding base model, standard contrastive-learning tooling.

Significance: The project's central, hardest-to-replicate technical asset.

8. Ingestion and Normalisation Component

Function: Pulls and standardises source data.

Responsibilities: Scheduled retrieval from retraction databases, preprint withdrawal snapshots, and linked-repository metadata; normalise each into a common record format.

Technology: Python, Vercel Cron Jobs, as described in the Deployment and Operations section.

Significance: Determines the actual size and quality of the bounded prototype corpus described in the Scope and Data Strategy sections.

9. Retrieval Component (Postgres and pgvector)

Function: Serves query-time retrieval for both researcher-facing modules.

Responsibilities: Index embeddings and failure-record metadata; serve nearest-neighbour and relational queries with acceptable latency.

Technology: Postgres, pgvector, hosted as Vercel Postgres.

Significance: The shared data layer both core modules depend on.

10. Reviewer Queue and Verification Interface

Function: Gives the recruited reviewer pool a working interface to confirm or reject flagged items.

Responsibilities: Present pending items with source context; capture confirm or reject decisions; feed decisions back into the Verification Exchange's scoring.

Technology: A lightweight authenticated view within the Next.js frontend, backed by the FastAPI backend.

Significance: The human half of the platform's confidence-calibration discipline, and the component the Timeline Risk section identifies as needing to be usable early.

11. API Gateway and Backend Service Layer (FastAPI)

Function: Exposes all platform functionality as a documented REST API.

Responsibilities: Route requests to the Graveyard, Citation Necromancer, and Verification Exchange; enforce request validation; generate the OpenAPI schema the frontend and enterprise integrations depend on.

Technology: Python, FastAPI.

Significance: The single integration point institutional and enterprise customers depend on, as described in the Technology Stack section.

12. AI Integration Component (Gemini API Interface)

Function: Manages all calls to Google's Gemini models.

Responsibilities: Handle request submission and response parsing for classification and explanation tasks; enforce the provider-abstraction interface described in the Technology Stack section; support fallback to a locally hosted alternative.

Technology: Python, the Gemini API.

Significance: The component most exposed to the AI Provider Risk described in the Risks and Caveats section, and the reason that risk's mitigation exists at all.

13. Deployment and CI/CD Pipeline (Vercel and GitHub Actions)

Function: Deploys and operates the platform.

Responsibilities: Build and deploy the frontend and the Python serverless backend; run the scheduled ingestion jobs; run tests before every deploy.

Technology: Vercel, GitHub Actions, as described in the Deployment and Operations section.

Significance: Keeps the team's operational burden inside what a small team can sustain, as discussed in the Deployment and Operations section.

Data Strategy

Bounded Source Selection for the Prototype

The prototype draws on openly licensed sources only. A public retraction database provides structured retraction records with stated reasons where available. Preprint server withdrawal notices, drawn from a defined snapshot rather than a live feed, provide the second failure signal. Public code repository metadata, restricted to repositories explicitly linked to a published or preprinted paper, provides the third, noisiest signal, filtered through the abandonment classifier described earlier.

Corpus Boundary for the Matching Stage

The Citation Necromancer's cross-domain matching is demonstrated on a deliberately bounded pair or small set of fields chosen for the availability of open, well-structured metadata, rather than across the entirety of scientific literature, which the Scope section states as an explicit exclusion for this phase.

Licensing and Attribution

Every source used carries an open licence permitting the derived use Lazarus makes of it, and the platform's documentation and interface disclose the provenance of every record shown to a user. No paywalled full-text content is scraped or reproduced; where a source's terms permit metadata and abstract-level use only, the platform respects that boundary even where fuller text might improve classification quality.

Commercialisation Strategy

Freemium Individual Tier

Individual researchers can access the Graveyard and Citation Necromancer with limited monthly queries at no cost, lowering the barrier to first use and letting the platform build a track record and word-of-mouth reputation within the academic community before asking any individual to pay. A paid individual tier lifts the query limit and adds saved-search and alert features for researchers who use the platform regularly.

Business-to-Consumer (B2C) Positioning

The paid individual tier is priced to be affordable on a personal or lab discretionary budget, positioned as a professional tool a researcher expenses or pays for personally, similar to a reference-manager subscription, rather than as a purchase requiring institutional approval.

Business-to-Business (B2B) Positioning

The institutional and enterprise tiers described in the Target Audience section are the primary commercial engine: university research offices and libraries purchase population-wide access, and corporate research and development departments purchase the enterprise tier for its API access and priority verification. These are B2B sales motions with a centralised purchasing decision, longer sales cycles, and materially higher contract values than the individual tier.

Strategic Partnerships

Partnerships with open-access preprint servers and retraction-tracking initiatives are a natural extension of the platform's reliance on their data, potentially formalising data-sharing arrangements that improve corpus coverage in exchange for attribution and, where appropriate, revenue sharing. Partnerships with university research-integrity offices, whose mandate already includes tracking retractions and negative results, are a natural institutional entry point beyond a pure subscription sale.

Scalable Revenue Streams

Revenue is designed to scale along two axes independently: the number of individual and institutional subscribers, and the depth of the enterprise offering as the mined corpus and matching quality mature. This dual scaling path is why the five- and ten-year outlook in the Budget and Return on Investment section shows enterprise revenue overtaking individual and institutional revenue only in the later years, once the corpus is large enough to make enterprise-grade reliability credible.

Cost Optimisation Strategy

Operating within the RM 100,000 ceiling, and sustaining a commercial service afterward, depends on keeping the platform's two largest variable costs, Gemini API calls and human verification time, from scaling linearly with usage.

Caching and Reuse of AI Outputs

Failure classifications and match explanations are cached permanently once generated for a given record pair, so the same retraction record or the same cross-domain match is never sent through the Gemini API twice. Because the mined corpus and the embedding index change far more slowly than query volume, this cache absorbs the majority of repeat queries at effectively zero marginal API cost.

Batch Rather Than Per-Query Classification

Failure classification runs as a scheduled batch process against newly ingested records, described in the Deployment and Operations section, rather than on demand per user query, so the cost of classifying the corpus is fixed and predictable regardless of how many researchers query it afterward.

Tiered Feature Access

Query limits on the free individual tier cap the platform's exposure to unpaid API usage while still letting every researcher experience the core product, and the paid tiers, where usage is materially higher, are exactly the tiers generating the revenue that funds that usage.

Lightweight Pre-Filtering Before AI Invocation

Simple, rule-based checks, such as confirming a query contains enough structured information to be worth matching, run before any Gemini API call or embedding lookup, so malformed or trivial queries are rejected at zero cost rather than consuming a paid API call.

Verification Queue Sizing

The Verification Exchange's confidence thresholds, described in the Lazarus Concept section, are set specifically to balance reviewer cost against output trustworthiness: routing too much output to human review would make the Human Verification Layer budget line unsustainable at scale, while routing too little would undermine the calibration claim central to this proposal's credibility.

Budget and Return on Investment

Budget Allocation Within the RM 100,000 Ceiling

The proposed budget allocates the full RM 100,000 ceiling across five categories, weighted toward the modelling work that is the project's core technical claim and toward the human-verification layer that protects its credibility.

Category	Allocation (RM)	Purpose
Compute and model API costs	25,000	Embedding fine-tuning compute, Gemini API calls for classification and match explanation
Engineering and development	40,000	Contract or team development effort to take the pipeline from prototype to a pilot-ready MVP
Human verification layer	15,000	Subject-matter reviewer time for the Verification Exchange's review queue during the pilot period
Data acquisition and licensing	10,000	Access to any bounded closed-metadata sources beyond fully open corpora, if required
Design, hosting, and deployment	7,000	Application layer hosting, interface design, and deployment infrastructure
Contingency	3,000	Unallocated reserve
Total	100,000	

Revenue Model

Lazarus is proposed as a subscription platform with three tiers. An individual researcher tier, priced to be affordable on a personal or lab discretionary budget, provides Graveyard and Citation Necromancer access with standard query limits. An institutional tier, sold to university research offices and libraries, provides unlimited access for a defined researcher population and is the primary near-term revenue driver. An enterprise tier, sold to corporate research and development departments, adds API access, private corpus integration, and priority

verification turnaround, at a materially higher price point reflecting the higher economic value a failed-experiment warning carries in a commercial R&D setting.

Five-Year Outlook (Year 1 to Year 5)

The following projection assumes a modest, deliberately conservative adoption curve: a small number of pilot institutional partners in Year 1, growing individual and institutional adoption through Year 3 as the mined corpus and match quality improve, and enterprise-tier adoption beginning only once the platform has an established institutional track record.

Year	Revenue (RM)	Operating Cost (RM)	Net (RM)
Year 1	20,000	120,000	(100,000)
Year 2	80,000	150,000	(70,000)
Year 3	220,000	220,000	0
Year 4	450,000	350,000	100,000
Year 5	700,000	500,000	200,000
Cumulative Years 1–5	—	—	130,000

Second Five-Year Outlook (Year 6 to Year 10)

By Year 6, the projection assumes the enterprise tier has begun contributing meaningfully, alongside continued institutional and individual growth, with operating costs growing more slowly than revenue as the fixed cost of the mined corpus and the trained embedding model is amortised across a larger user base.

Year	Revenue (RM)	Operating Cost (RM)	Net (RM)
Year 6	1,000,000	650,000	350,000
Year 7	1,400,000	850,000	550,000
Year 8	1,850,000	1,100,000	750,000
Year 9	2,300,000	1,350,000	950,000
Year 10	2,800,000	1,600,000	1,200,000
Cumulative Years 6–10	—	—	3,800,000

A Note on These Projections

These figures are planning-stage estimates built from stated, conservative assumptions about subscriber counts and price points, not audited forecasts. Their purpose is to demonstrate that the team has thought rigorously about a path to financial sustainability beyond the initial funding period, not to promise a specific financial outcome.

Key Investment Metrics (Five-Year Basis)

Translating the five-year projection above into the standard investment metrics: at an assumed 10% discount rate, the project's Net Present Value (NPV) over the five-year period is approximately RM 43,700. Return on Investment is 130% on an undiscounted, nominal basis and approximately 44% on an NPV basis. The Payback Period is approximately 4.3 years, reflecting the deliberately conservative assumption that revenue trails cost until the institutional base matures. The Internal Rate of Return (IRR) is approximately 19%. These figures use a standard 10% discount rate assumption; a different rate or adoption curve would shift them, and they carry the same planning-stage caveat as the rest of this section.

Metric	Value
Net Present Value (NPV)	RM 43,700
Return on Investment (undiscounted)	130%
Return on Investment (NPV-based)	44%

Metric	Value
Payback Period	~4.3 years
Internal Rate of Return (IRR)	~19%
Funding Requested	RM 100,000

Development Challenges

Several challenges are expected during development, distinct from the ongoing risks discussed in the following section.

Defining a labelled dataset for the failure classifier is harder than it first appears, since “why did this fail” is often stated ambiguously or not at all in a retraction notice, requiring a deliberate annotation effort against the fixed failure taxonomy before the classifier can be trained with confidence. The team's plan is to bootstrap this effort from the clearest, most explicit cases first, retraction notices with an unambiguous, single stated cause, before expanding to the sparser, more ambiguous withdrawal notices and repository signals where the taxonomy has to be applied with more inference and less certainty.

Training a structural, rather than topical, embedding model is the project's central technical challenge precisely because no existing labelled dataset of methodologically-similar cross-field paper pairs exists to train against. The team expects to construct an initial contrastive training set through known historical cases of cross-field method transfer, documented in the history-of-science literature, and synthetic pair construction, then to validate and expand it using the Verification Exchange's own reviewer pool once the pipeline is running.

Assembling a corpus that is simultaneously bounded enough to be demonstrable within the budget, and broad enough to make the cross-domain matching claim credible to a sceptical evaluator, is a scope-balancing challenge running throughout the build, addressed directly in the Scope section above.

Recruiting an initial reviewer pool for the Verification Exchange before the platform has any track record is a cold-start challenge common to any system with a human-in-the-loop quality layer, expected to be addressed initially through the team's own academic networks before broader recruitment becomes viable. Early reviewers are offered acknowledgement and early access to paid-tier features rather than direct payment in this earliest phase, with the Human Verification Layer budget line in the Budget and Return on Investment section only covering paid reviewer time once the pilot institutional partnerships described in the Roadmap section are in place.

Distinguishing genuine methodological failure from retractions driven by research misconduct is a challenge in its own right, not a solved pre-processing step. Fraud, data fabrication, and plagiarism account for a meaningful share of all retractions, and conflating a misconduct case with an honest methodological dead end would misrepresent the original authors in a way that carries real reputational consequences, addressed further in the Risks and Caveats section. Building a screening step reliable enough to make this distinction, rather than treating every retraction as equivalent evidence of a failed approach, is expected to require its own dedicated classification pass, informed by signals such as whether a misconduct investigation is publicly referenced, run separately from and prior to the failure-cause classification the Graveyard's functional description sets out.

Setting the Verification Exchange's confidence thresholds conservatively at launch is the right starting point, but the right threshold is not a value to be fixed once and left alone. As confirmed and rejected reviewer decisions accumulate, the thresholds should be recalibrated to reflect what the data actually supports, and doing this without destabilising the confidence guarantees researchers have come to rely on, tightening or loosening categories inconsistently as more evidence arrives, is an ongoing methodological challenge rather than a one-time design decision.

Open, well-structured retraction and preprint metadata is not evenly available across scientific fields: some disciplines have mature preprint cultures and active retraction-tracking communities, and others do not. This means the Graveyard and the Citation Necromancer will, at launch, perform noticeably better for some fields than others, purely as a function of data availability rather than any property of the underlying method, and the team commits to disclosing this unevenness explicitly in the product itself, addressed further in the Risks and Caveats section, rather than presenting uniform coverage the corpus does not yet have.

Risks and Caveats

Cold-Start Value Risk

The Citation Necromancer's value depends on the size and field-diversity of the corpus it draws on, and a small prototype corpus will produce a proportionally small number of genuinely compelling cross-domain matches. This risk is acknowledged directly rather than hidden, and the Scope section defines a deliberately bounded corpus specifically so the prototype's claims match what a bounded corpus can actually support. The mitigation is one of framing as much as engineering: the pitch and the demonstration are built around a small number of genuinely striking matches within the bounded corpus, rather than an implicit promise of comprehensive coverage the prototype cannot yet deliver.

False Confidence Risk

Because a wrong claim actively wastes a researcher's time and trust, the confidence thresholds feeding the Verification Exchange are set conservatively, favouring routing more output to human review over surfacing borderline output directly, even at the cost of a slower, smaller stream of system-verified results in the early period. This risk compounds with the Misconduct-Classification Risk below wherever a failure record's cause is ambiguous, which is why both risks share the same conservative-default mitigation rather than being treated as separate problems requiring separate thresholds.

Misconduct-Classification Risk

Because a meaningful share of retractions stem from research misconduct rather than genuine methodological failure, described in the Development Challenges section, misclassifying a misconduct-driven retraction as an honest methodological failure, or the reverse, carries real reputational stakes for the named authors of the original work. The mitigation is a conservative default rather than a claim of a perfect screen: any record where the misconduct-versus-failure distinction itself falls below a high confidence threshold is withheld from the Graveyard entirely rather than surfaced with a low-confidence tag, since the reputational cost of a wrong classification here is higher than the cost of a missed result.

AI Provider Risk

Reliance on Google's Gemini models for classification and match explanation introduces exposure to that provider's pricing, availability, and rate limits. The provider-abstraction approach described in the Technology Stack section mitigates this by keeping the calling code independent of any single provider, so a locally hosted open-weight model can be substituted if Gemini's terms, pricing, or availability change.

Data Licensing Risk

The prototype depends on the continued open licensing terms of its source databases. Should any source materially restrict its terms, the affected slice of the corpus would need to be replaced or reduced, which the bounded-corpus approach in the Data Strategy section is designed to make manageable rather than catastrophic.

Uneven Coverage and Field Bias Risk

The corpus's field coverage is uneven at launch, described in the Development Challenges section, and a researcher in a sparsely covered field could reasonably misread an empty result set as evidence that no one has tried their approach before, when the more accurate reading is that the corpus simply does not yet cover their field well. The interface is designed to state coverage limitations explicitly alongside any query in an under-represented field, rather than let a silent empty result imply a false negative.

Adoption and Behaviour-Change Risk

Lazarus asks researchers to adopt a new step in their workflow, checking a system before beginning work, which is a behaviour-change ask rather than a passive product. Selling through university research offices rather than relying solely on individual adoption is intended to partially de-risk this by embedding the platform at the point where new researchers are onboarded.

Competitive Risk

Existing, well-resourced literature-AI platforms could add failure-mining or cross-domain matching features of their own. The team's mitigating position is that the structural embedding approach described in this proposal is a genuine, non-trivial piece of machine-learning work that a feature addition to an existing summarisation-focused product is unlikely to replicate quickly or well.

Timeline Risk

Six months is a tight window for training a novel embedding model, recruiting institutional pilot partners, and launching paid subscription tiers, and any one of the three slipping would compress the others. The mitigation is twofold: the scope discipline described earlier in this proposal keeps the technical build small enough to fit the window, and pilot partner recruitment, the slowest and least controllable step, begins in parallel during the build phase rather than only after it, so an institutional partner's own approval timeline does not sit entirely on the plan's critical path.

Roadmap

The team has committed to completing the project, from this proposal through commercial launch, within six months. This is achievable specifically because of the scope discipline set out earlier in this document: a bounded corpus, a narrow verification role, and no attempt at comprehensive real-time coverage keep the six-month plan realistic rather than aspirational. The longer-term roadmap beyond launch continues into the five- and ten-year outlook set out in the Budget and Return on Investment section.

Phase 0 — Concept Validation (Pre-Project)

The phase covering the period up to and including this proposal's submission focuses on validating the core claims well enough to present them credibly: a small proof-of-concept corpus demonstrating the Graveyard's classification approach, an initial version of the structural embedding model trained on a hand-assembled contrastive set, and a working illustration of the Verification Exchange's confidence-routing logic.

- Assemble the hand-labelled contrastive training set from documented historical cases of cross-field method transfer
- Build a minimal classifier prototype against a small sample of retraction records
- Sketch the confidence-routing logic in a lightweight script rather than a live system

Phase 1 — Minimum Viable Pipeline (Months 1 to 2)

The first two months take the proof-of-concept components to a working end-to-end pipeline against the bounded prototype corpus, with a basic web interface exposing the Graveyard and Citation Necromancer to a small internal test group. Outreach to prospective pilot institutional partners begins during month two, run in parallel with the final build work rather than afterward, since institutional approval timelines are the slowest and least controllable part of the plan.

- Weeks 1–2: Repository and infrastructure setup — Next.js and TypeScript scaffold, FastAPI backend scaffold, Vercel project, Vercel Postgres with the pgvector extension enabled
- Weeks 3–4: Ingestion pipeline — connect to the chosen retraction database and preprint withdrawal snapshot, build the normalisation layer, run the first batch ingestion against the bounded corpus
- Weeks 5–6: Failure classifier — build and validate the failure-cause taxonomy classifier against the Gemini API, wire up the Graveyard's query interface end-to-end
- Weeks 7–8: Structural embedding model — fine-tune the initial version against the hand-assembled contrastive set, wire up the Citation Necromancer's retrieval end-to-end; pilot partner outreach begins in parallel

Phase 2 — Pilot with Institutional Partners (Months 3 to 4)

The next two months onboard the pilot university partners recruited during Phase 1, using their researcher populations both as early users and as the initial Verification Exchange reviewer pool, and use pilot feedback to refine confidence thresholds and the failure taxonomy.

- Weeks 9–10: Onboard the first pilot university partners and establish the initial Verification Exchange reviewer pool from their researcher population
- Weeks 11–12: Collect pilot usage data and run the first round of confidence-threshold calibration against real reviewer feedback
- Weeks 13–14: Refine the failure taxonomy and the structural embedding model based on pilot corrections
- Weeks 15–16: Harden the platform based on pilot feedback — usability fixes and tightened confidence-routing thresholds

Phase 3 — Commercial Launch (Months 5 to 6)

The final two months of the six-month plan launch the individual and institutional subscription tiers, expand corpus coverage beyond the initial bounded scope, and begin the revenue trajectory set out in the five-year projection.

- Weeks 17–18: Build the subscription and billing layer for the individual and institutional tiers
- Weeks 19–20: Expand corpus coverage beyond the initial bounded scope, guided by which fields the pilot showed the most demand for
- Weeks 21–22: Public launch — marketing through the pilot partners' networks, opening the individual freemium tier publicly
- Weeks 23–24: Post-launch monitoring and the first cost-versus-budget review

Phase 4 — Enterprise Expansion and Scale (Year 2 Onward)

Beyond the six-month build and launch, and aligned with the growth curve set out in the five- and ten-year outlook, this phase introduces the enterprise tier for corporate research and development customers and expands the structural embedding model's field coverage.

Phase	Focus	Primary Milestone
Phase 0 (Pre-Project)	Concept Validation	Working proof-of-concept and this proposal
Phase 1 (Mo. 1–2)	Minimum Viable Pipeline	End-to-end pipeline live; pilot outreach begins
Phase 2 (Mo. 3–4)	Pilot	Institutional pilot partners onboarded
Phase 3 (Mo. 5–6)	Commercial Launch	Subscription tiers live, revenue begins
Phase 4 (Year 2+)	Enterprise Expansion	Enterprise tier live, corpus scaled

Conclusion

Lazarus is proposed as a response to “Accelerating Scientific Discovery with Intelligence Research Technology” that reads acceleration as the avoidance of redundant failure and the deliberate manufacture of cross-domain analogy, rather than as faster literature search, an already well-served category. The platform is built around three stages sharing one corpus and one verification discipline: a Mining stage that converts the literature's scattered failure signal into a searchable resource, a Matching stage built on a structural rather than topical embedding model that is the project's central technical claim, and a Verification stage that keeps every claim the system makes calibrated and honest.

This proposal has set out the vision and deliberately bounded scope chosen for an open theme, the problems the team identified for itself in the absence of a stated brief, the objectives the project pursues, the architecture and functional design of each pipeline stage, the technology and data choices behind them, the target audience, a budget proposal within the stated RM 100,000 ceiling together with a conservatively modelled ten-year financial outlook, the development challenges and risks the team expects honestly, and a six-month roadmap to commercial launch that continues into the longer-term enterprise expansion set out in the financial outlook.

The deeper case for Lazarus is that a scientific literature which remembers only what worked is discarding most of the effort that has ever been spent on it. A system that resurrects the rest — honestly, calibrated, and at a scale no individual researcher could match by reading alone — is worth building whether or not this particular theme rewards it.